

Figure 2: Step 2 of the transaction flow

These RW sets are then signed by the endorsing peer, and returned to the client application to be used in future steps of the transaction flow.

This is the second step of the transaction flow, when endorsers simulate transactions, generate RW sets, and return the signed RW sets back to the client application. Endorsing peers must hold smart contracts in order to simulate the transaction proposals.

Transaction flow (Step 3)

The application then submits the endorsed transaction and the RW sets to the ordering service. Ordering happens across the network, in parallel with endorsed transactions and RW sets submitted by other applications.

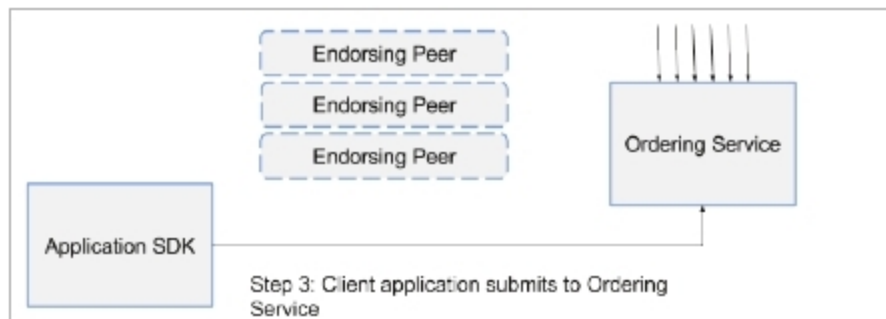


Figure 3: Step 3 of the transaction flow

Transaction flow (Step 4)

The ordering service takes the endorsed transactions and RW sets, orders this information into a block, and delivers the block to all committing peers. This is Step 4 of the transaction flow, where the orderer sends the ordered transactions in a block to all committing peers.

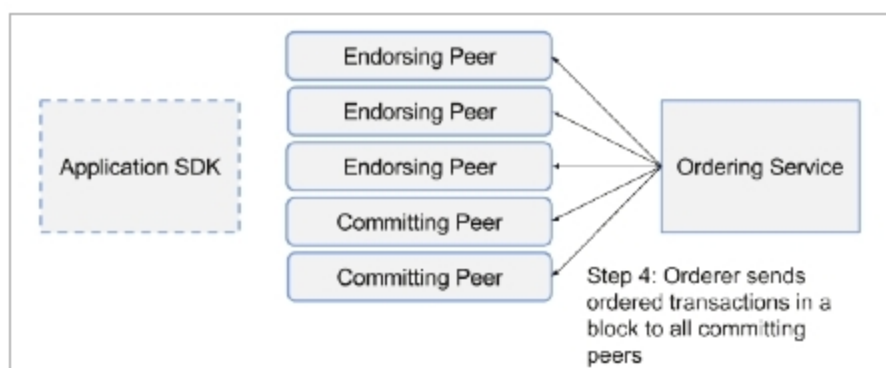


Figure 4: Step 4 of the transaction flow

The ordering service, which is made up of a cluster of orderers, does not process transactions, smart contracts, or maintain the shared ledger. It only accepts the endorsed transactions and specifies the order in which those

transactions will be committed to the ledger. The Fabric v1.0 architecture has been designed such that the specific implementation of 'ordering' (Solo, Kafka and BFT) becomes a pluggable component. The default ordering service for Hyperledger Fabric is Kafka. Therefore, the ordering service is a modular component of Hyperledger Fabric.

Transaction flow (Step 5)

The committing peer validates the transaction by checking to make sure that the RW sets still match the current world state—specifically, that the read data that existed when the endorsers simulated the transaction is identical to the current world state. When the committing peer validates the transaction, the transaction is written to the ledger, and the world state is updated with the write data from the RW set.

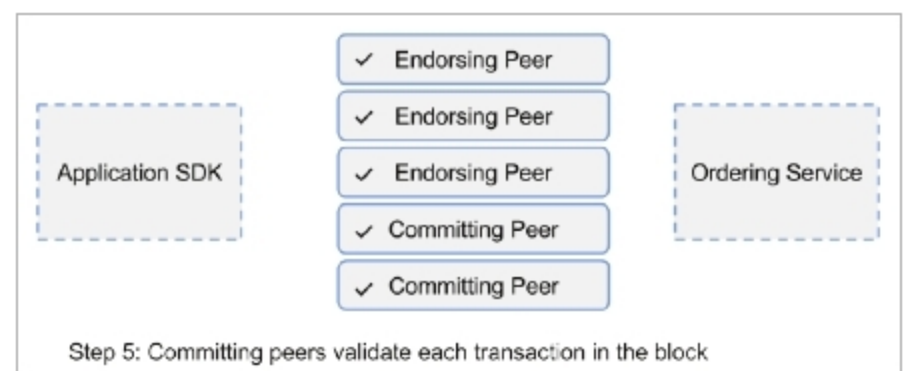


Figure 5: Step 5 of the transaction flow

This is the fifth step of the transaction flow, where committing peers validate each transaction in the block. If the transaction fails, that is, if the committing peer finds that the RW set does not match the current world state, the transaction ordered into a block will still be included in that block, but it will be marked as invalid, and the world state will not be updated. Committing peers are responsible for adding blocks of transactions to the shared ledger and updating the world state. They may hold smart contracts, but it is not a requirement.

Transaction flow (Step 6)

Lastly, the committing peers asynchronously notify the client application of the success or failure of the transaction. Applications will be notified by each committing peer.

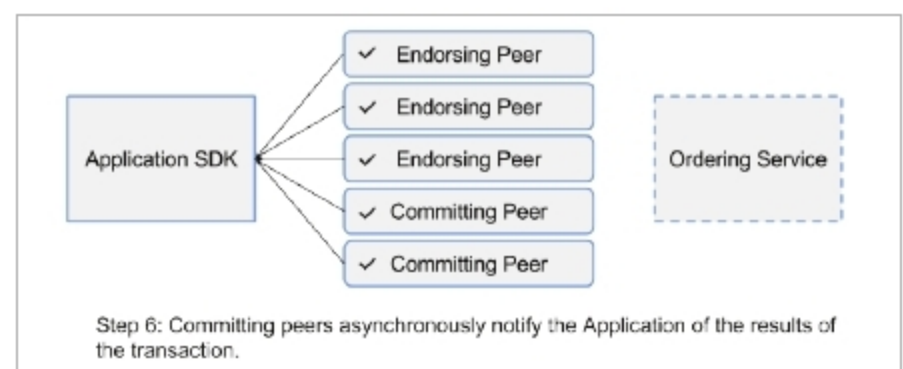


Figure 6: Step 6 of the transaction flow

Chaincode

In Hyperledger Fabric, chaincode is the 'smart contract' that runs on the peers and creates transactions. More broadly, it enables